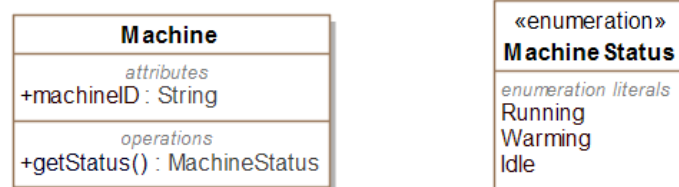


Análisis y Diseño con Patrones

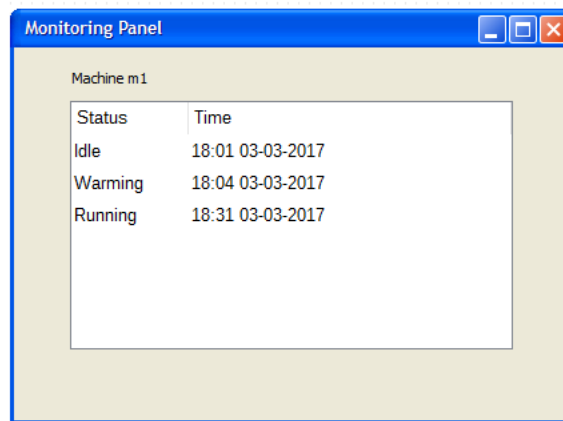
PEC 2: Patrones de Diseño y Asignación de Responsabilidades

Pregunta 1 (20%)

Una máquina, en un primer paso para llegar a ser un sistema ciberfísico, dispone de un sistema *embedded* para el cual se dispone de la librería de clases de bajo nivel que se muestra en el siguiente diagrama:



Nos piden que propongamos un diseño de software, estando autorizados a refactorizar los elementos que consideremos oportunos, para que terceros desarrolladores puedan añadir funcionalidades basándose en un mecanismo de subscripción y recepción de notificaciones de cambios de estado de la máquina con el fin de ser almacenados, tratados o presentados en una interfaz gráfica, como por ejemplo, un panel de monitorización conectado al sistema *embedded* como el de la siguiente figura:



Status	Time
Idle	18:01 03-03-2017
Warming	18:04 03-03-2017
Running	18:31 03-03-2017

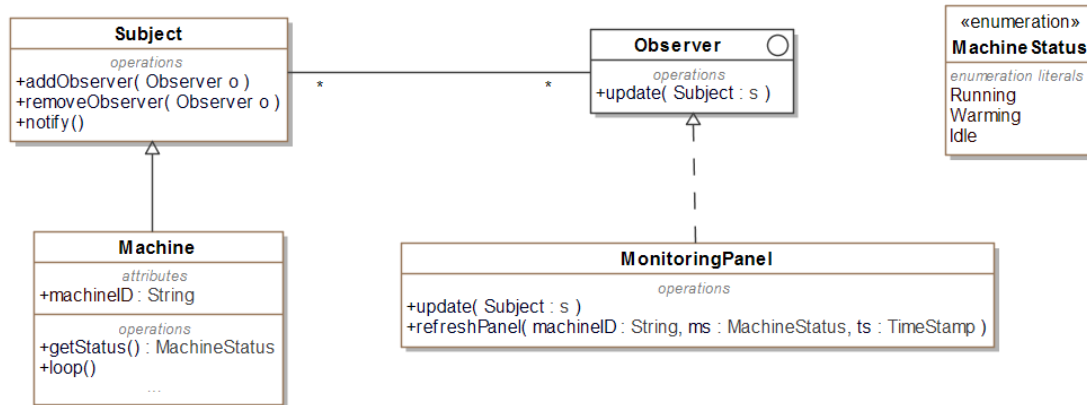
Se pide también que el diseño permita preservar el principio de diseño abierto-cerrado sobre el objeto notificante.

- ¿Qué patrón de diseño propondrías para este caso? Justifica brevemente la respuesta.
- Para la generación de eventos de notificación se ha decidido seguir un esquema de *polling* (muestreo cíclico) sobre la máquina cada t segundos, y en el caso de que la lectura de un estado sea distinta que la lectura anterior entonces se debe notificar dicho cambio. Realizad el diagrama de clases aplicando el patrón de diseño propuesto teniendo en cuenta que la información que se pide para cada cambio de estado de la máquina es la tupla $\langle \text{machineID}, \text{machineStatus}, \text{timestamp} \rangle$, indicando las transformaciones respecto el modelo original. Definid los atributos y operaciones necesarias para implementar el comportamiento descrito y considerad que ya existe una clase **MonitoringPanel** para presentar los datos.

Solución

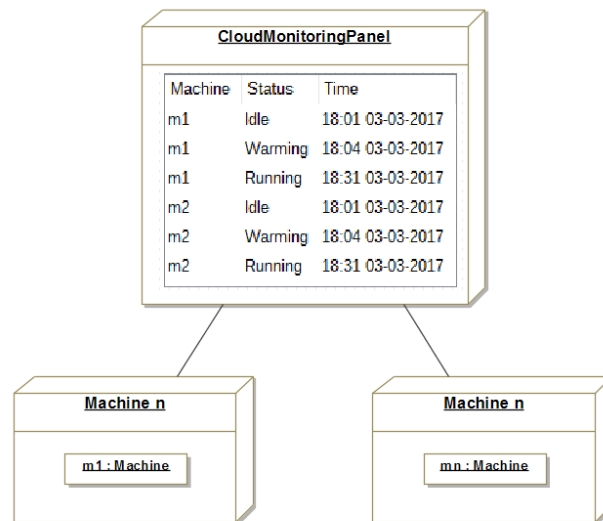
- Observador es el patrón adecuado para el diseño de soluciones en las que el cambio de estado de un objeto provoca notificaciones a otros objetos. El patrón contempla un mecanismo de desacoplamiento entre los objetos notificantes y los objetos notificados de modo que los primeros no tengan que modificarse a medida que el sistema vaya incorporando nuevas responsabilidades mediante la ampliación del conjunto de los segundos. Ello permite preservar el principio abierto-cerrado gracias a lo cual el sistema será extensible por parte de terceros desarrolladores.
- Implementaremos una operación denominada **Machine::loop()** que cada t segundos

invocará a `getStatus()` y comprobará si `machineStatus` es distinto al obtenido en la invocación anterior, en cuyo caso se invocará a la operación `notify()`. El diagrama de clases resultante de aplicar el patrón Observador es:



Pregunta 2 (20%)

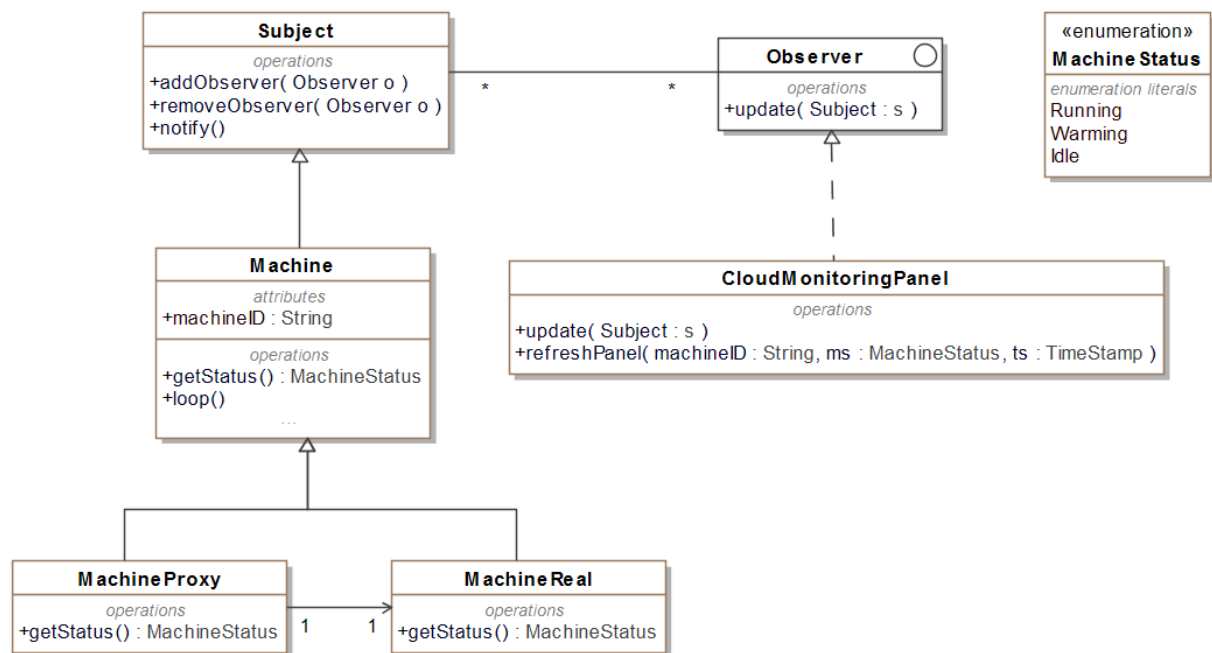
Supongamos que una fábrica dispone de n máquinas, cada una de ellas conectada a la nube mediante su sistema *embedded* y provista de la librería de bajo nivel descrita en la pregunta anterior. Se nos pide que diseñemos una solución que permita crear instancias de **Machine** tanto en el sistema *embedded* como en la nube, de modo que en ambos casos el acceso al objeto que conecta con el sistema físico sea transparente, con el fin de diseñar un panel de monitorización conjunta en la nube aprovechando en lo posible el diseño de la pregunta 1. Dicho panel se muestra en el siguiente diagrama de despliegue UML:



- ¿Qué patrón de diseño propondrías para este caso? Justifica brevemente la respuesta.
- Realizad el diagrama de clases aplicando el patrón de diseño propuesto, indicando las transformaciones respecto el modelo original. Definid los atributos y operaciones necesarias para implementar el comportamiento descrito

Solución

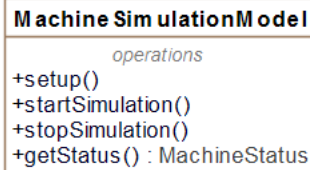
- El patrón Proxy, en su variante Proxy remoto, es adecuado para el diseño de la solución al problema planteado puesto que permitirá disponer desde la nube acceso a las instancias de **Machine** de los sistemas *embedded* de forma transparente, es decir, como si fueran locales. La instancia de **CloudMonitoringPanel** pueda añadirse como observadora de las diversas instancias de **Machine** en la nube.
-



La instancia de **MachineReal** se crea en el sistema embedded, mientras que las de **MachineProxy** y las del resto de clases se crean en la nube. Al invocar **Machine::getStatus()** en la nube en realidad se ejecuta **MachineProxy::getStatus()** que a su vez realiza una invocación remota a **MachineReal::getStatus()**.

Pregunta 3 (30%)

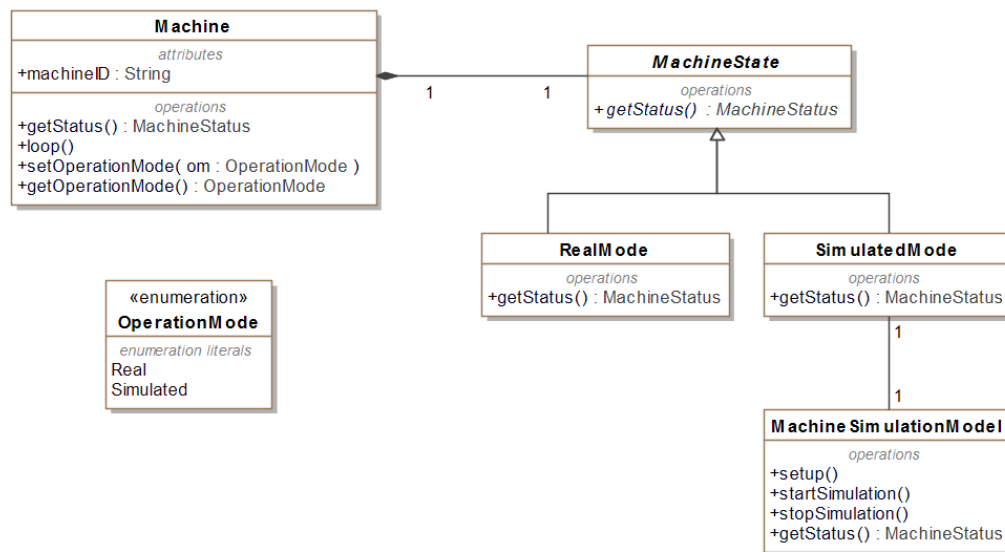
Nos piden que, con el fin de disponer de entornos de entrenamiento o bien para realizar pruebas, las máquinas puedan funcionar en dos modos de operación, por una parte en modo real y por otra en modo simulado. Cuando están en modo real la operación `getStatus()` funciona tal y como se ha descrito en las preguntas anteriores, y cuando está en modo simulado el sistema *embedded* ejecuta un modelo de simulación de modo que se pueden probar las funcionalidades de **Machine** sin que la máquina esté realmente en funcionamiento. Para ello disponemos del siguiente modelo de simulación:



- ¿Qué patrón de diseño propondrías para este caso? Justifica brevemente la respuesta.
- Realizad el diagrama de clases aplicando el patrón de diseño propuesto, indicando las transformaciones respecto el modelo original. Definid los atributos y operaciones necesarias para implementar el comportamiento descrito
- ¿Quién debería ser el responsable de crear las instancias de **MachineSimulationModel**?
¿Qué patrón habría que aplicar para garantizar que siempre haya una instancia y sólo una de **MachineSimulationModel** disponible?

Solución

- El patrón de diseño Estado es el adecuado para este caso, puesto que el comportamiento de la máquina varía en función de su estado. Para ello distinguiremos dos estados a los que haremos corresponder los dos modos de operación: El estado **RealMode** y el estado **SimulatedMode**.
-



La lógica de bajo nivel de **Machine::getStatus()** ha pasado a **RealMode::getStatus()**.

La solución dada no tiene en cuenta la aplicación del patrón Proxy de la pregunta 2, pero si hubiera que tenerse en cuenta habría que utilizar **MachineReal** en lugar de **Machine**.

- c) Según es patrón de asignación de responsabilidades Creador, **SimulatedMode** es la clase que utiliza más estrechamente objetos de **MachineSimulationModel**, y según este criterio a ella le corresponderá crear sus instancias. Para garantizar que siempre haya una instancia y sólo una de **MachineSimulationModel**, independiente de que ésta exista o no previamente, la aplicación del patrón Singleton en el proceso de creación resuelve este problema.

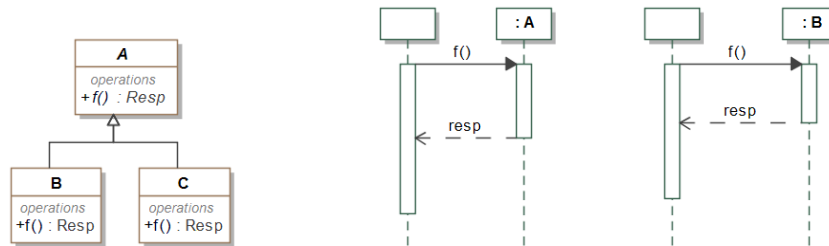
Pregunta 4 (30%)

Para ilustrar la dinámica de los mecanismos de los patrones de diseño utilizados en esta PEC, nos piden que elaboremos un diagrama de secuencia de las invocaciones de **getStatus()** para cada uno de los siguientes escenarios, considerando las respuestas a las preguntas 1, 2 y 3:

- Invocación desde el sistema *embedded* local en modo real
- Invocación desde el sistema *embedded* local en modo simulado

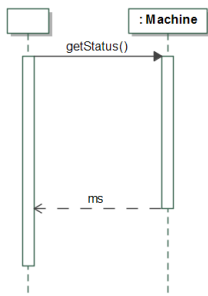
- Invocación desde la nube

Para realizar los diagramas de secuencia, suponed que ya están creadas las instancias de los objetos implicados, y para ilustrar los comportamientos polimórficos se puede descomponer el diagrama de secuencia tal y como se muestra en el siguiente ejemplo:



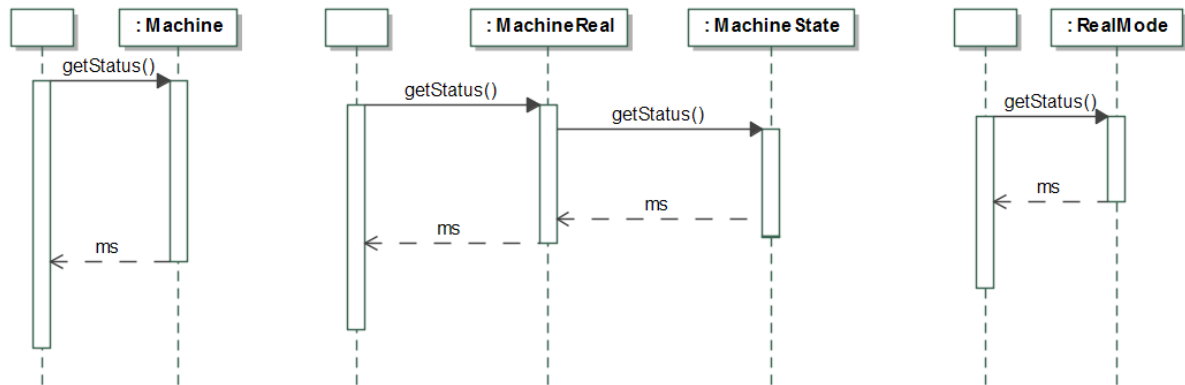
Solución

En la solución se han tenido en cuenta la aplicación de los patrones Proxy y Estado. Com que el sistema visto desde fuera no varía según el estado, ello implica que la invocación externa a `getStatus()` es transparente al modo de operación:

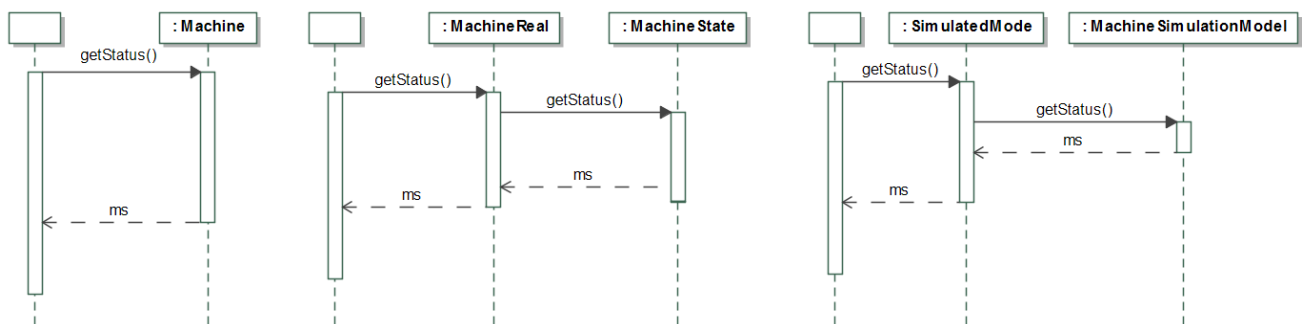


Para mayor claridad se repiten partes comunes de los diagramas de secuencia aunque realmente no haría falta en un repositorio UML integrado como MAgicDraw en que podrían estar enlazados entre sí.

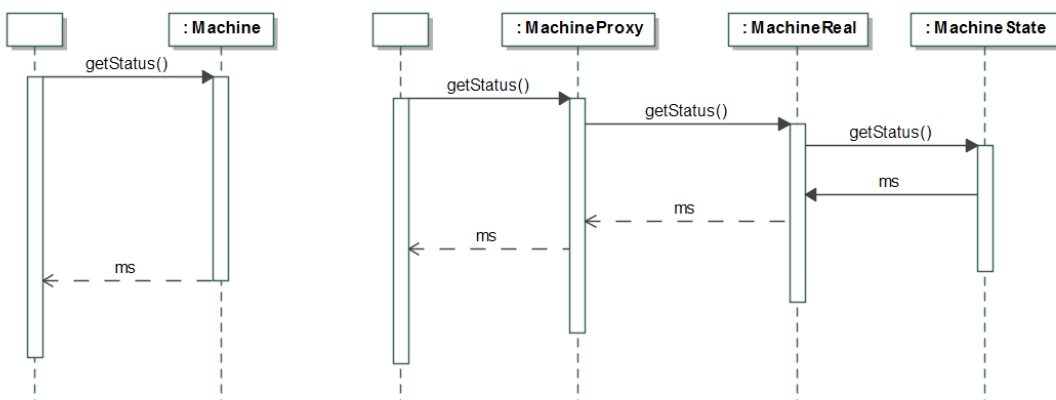
Invocación desde el sistema *embedded* local en modo real:



Invocación desde el sistema *embedded* local en modo simulado:



Invocación desde la nube:



A partir de `: MachineState` el comportamiento sería el mismo que el de los escenarios anteriores.